demo.py

```python
"""
Machine Learning Classification System
Dataset: Iris (loaded from scikit-learn; internet not required)
Run: python main.py
"""

import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier

OUTPUT_DIR = "outputs"
os.makedirs(OUTPUT_DIR, exist_ok=True)


def main():
    print("=" * 60)
    print("MACHINE LEARNING CLASSIFICATION SYSTEM")
    print("=" * 60)

    # 1. Load dataset
    iris = load_iris(as_frame=True)
    X = iris.data
    y = iris.target
    target_names = iris.target_names

    df = X.copy()
    df["species"] = y.map(dict(enumerate(target_names)))

    print("\n1. DATASET OVERVIEW")
    print(f"Rows: {df.shape[0]} | Columns: {df.shape[1]}")
    print("\nFirst 5 rows:")
    print(df.head().to_string(index=False))
    print("\nMissing values per column:")
    print(df.isnull().sum().to_string())
    print("\nSpecies counts:")
    print(df["species"].value_counts().to_string())

    # 2. Train/test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42, stratify=y
    )
```

```
52
53      # 3. Define models
54      models = {
55          "Logistic Regression": Pipeline([
56              ("scaler", StandardScaler()),
57              ("model", LogisticRegression(max_iter=1000, random_state=42))
58          ]),
59          "K-Nearest Neighbors": Pipeline([
60              ("scaler", StandardScaler()),
61              ("model", KNeighborsClassifier(n_neighbors=5))
62          ]),
63          "Decision Tree": DecisionTreeClassifier(max_depth=4, random_state=42)
64      }
65
66      results = []
67      trained_models = {}
68
69      # 4. Train and evaluate
70      print("\n2. MODEL RESULTS")
71      for name, model in models.items():
72          model.fit(X_train, y_train)
73          predictions = model.predict(X_test)
74          accuracy = accuracy_score(y_test, predictions)
75          results.append({"Model": name, "Accuracy": accuracy})
76          trained_models[name] = model
77
78          print(f"\n{name}: {accuracy:.2%} accuracy")
79          print(classification_report(
80              y_test, predictions, target_names=target_names, zero_division=0
81          ))
82
83          # Save confusion matrix
84          cm = confusion_matrix(y_test, predictions)
85          plt.figure(figsize=(6, 4))
86          sns.heatmap(
87              cm, annot=True, fmt="d", cmap="Blues",
88              xticklabels=target_names, yticklabels=target_names
89          )
90          plt.title(f"Confusion Matrix - {name}")
91          plt.xlabel("Predicted label")
92          plt.ylabel("True label")
93          plt.tight_layout()
94          safe_name = name.lower().replace(" ", "_").replace("-", "")
95          plt.savefig(os.path.join(OUTPUT_DIR, f"{safe_name}_confusion_matrix.png"),
    dpi=150)
96          plt.close()
97
98      # 5. Compare accuracy
99      results_df = pd.DataFrame(results).sort_values("Accuracy", ascending=False)
100     results_df["Accuracy (%)"] = (results_df["Accuracy"] * 100).round(2)
101     results_df.to_csv(os.path.join(OUTPUT_DIR, "model_comparison.csv"), index=False)
102
103     print("\n3. MODEL COMPARISON")
104     print(results_df[["Model", "Accuracy (%)"]].to_string(index=False))
```

```
105
106     plt.figure(figsize=(8, 5))
107     sns.barplot(data=results_df, x="Accuracy (%)", y="Model")
108     plt.title("Classification Model Accuracy Comparison")
109     plt.xlabel("Accuracy (%)")
110     plt.xlim(0, 100)
111     plt.tight_layout()
112     plt.savefig(os.path.join(OUTPUT_DIR, "model_comparison.png"), dpi=150)
113     plt.close()
114
115     # 6. Example prediction using the best model
116     best_name = results_df.iloc[0]["Model"]
117     best_model = trained_models[best_name]
118     sample = [[5.1, 3.5, 1.4, 0.2]]  # sepal length, sepal width, petal length, petal
    width
119     prediction = best_model.predict(sample)[0]
120
121     print("\n4. SAMPLE PREDICTION")
122     print("Measurements: sepal length=5.1, sepal width=3.5, petal length=1.4, petal
    width=0.2")
123     print(f"Best model: {best_name}")
124     print(f"Predicted species: {target_names[prediction]}")
125
126     print(f"\nCharts and results saved in the '{OUTPUT_DIR}' folder.")
127     print("Project completed successfully!")
128
129
130 if __name__ == "__main__":
131     main()
132
133
```